Assignment No. 5

Name: Piyush Rajendra Waghmare

PRN: 123B1B284

Subject: FSDL Lab

Aim : Create a simple functional college website / travel agency website / insurance website using Node.js and Express using any NoSQL data set like MongoDB if required.

Project Objectives :

The primary objectives of building this full-stack web application include:

- **Demonstrate Client-Server Communication:** To successfully establish a connection between a frontend user interface and a backend server using RESTful API principles.
- **Implement Database Integration:** To transition from static, hard-coded data to dynamic data retrieved from a NoSQL database (MongoDB).
- **Develop CRUD Capabilities:** To implement 'Create', 'Read', and 'Update' functionalities through an administrative form that directly modifies database records.
- **Master State Management:** To use React hooks (useState, useEffect) to seamlessly handle asynchronous data fetching and UI re-rendering without page reloads.

Project Outcomes

Upon completing this project, the following outcomes were achieved:

- **Full-Stack Proficiency:** Successfully integrated React.js, Node.js, Express, and MongoDB into a cohesive, functional application.
- **Data Modeling:** Gained hands-on experience designing NoSQL database schemas using Mongoose.
- **Responsive UI Design:** Created a modern, accessible, and visually appealing user interface utilizing advanced inline CSS and conditional rendering.
- **Error Handling:** Implemented robust try-catch blocks and loading states to ensure the application remains stable even if the database connection is delayed or fails.

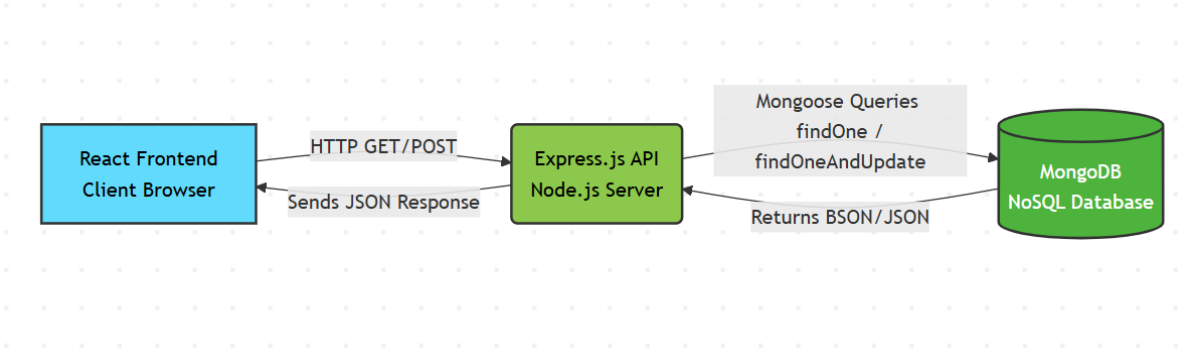
Tech Stack Used

Component	Technology	Description / Purpose
Frontend	React.js	Builds the dynamic user interface and manages component state.
Backend	Node.js	JavaScript runtime environment executing the server-side code.
Web Framework	Express.js	Handles HTTP requests, CORS, and defines the REST API routes.
Database	MongoDB	NoSQL document database used to store the college's information.
ODM / Tooling	Mongoose	Object Data Modeling library providing schema validation for MongoDB.

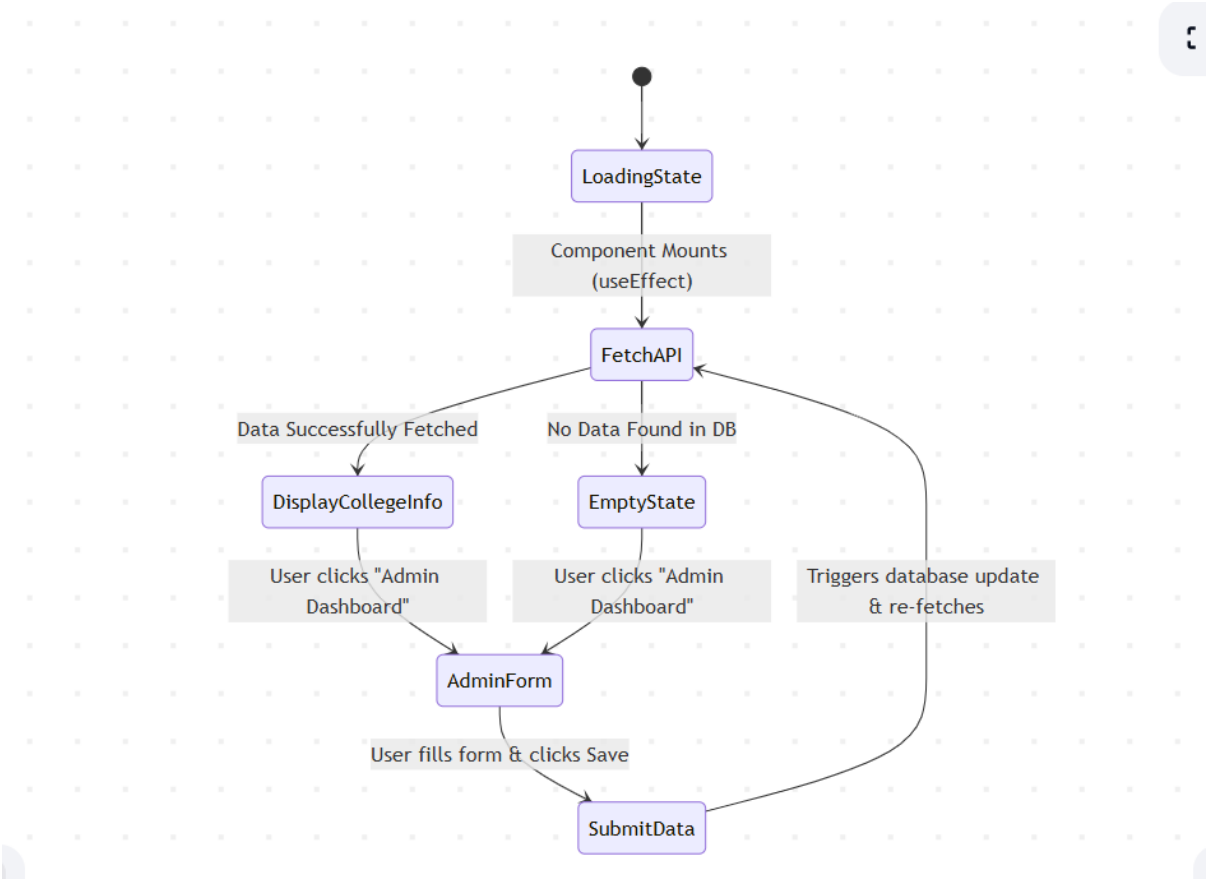
Theory Concepts & Architecture

This project follows a classic **3-Tier Architecture** (Presentation Layer, Application Layer, Data Layer). Instead of the frontend talking directly to the database (which is a major security risk), it communicates with an Express API, which acts as the middleman.

A. System Architecture Diagram This diagram shows how the three main technologies interact.



B. Data Flow & State Management Diagram This diagram illustrates the logical flow within the React application when a user switches between viewing data and editing data.



Dataset Used

Because MongoDB is a NoSQL database, the dataset is not stored in rigid rows and columns like SQL, but rather in flexible JSON-like documents (BSON). The primary dataset used for this project consists of a single document structure that holds the dynamic elements of the website.

MongoDB Document Structure (Mongoose Schema):

{

"_id": "ObjectId('64f1a2b3c4d5e6f7a8b9c0d1')",

"name": "Global Tech University",

"aboutText": "Empowering the leaders of tomorrow through innovation and excellence. Founded in 1995.",

"coursesOffered": [

"Computer Science Engineering",

"Mechanical Engineering",

"Business Administration"

],

"contactEmail": "admissions@globaltech.edu",
"__v": 0
}

Code Implementation

---→ React(fronend) App.jsx

```
import React, { useState, useEffect } from 'react';

function App() {
  const [collegeData, setCollegeData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [isEditing, setIsEditing] = useState(false);

  const [formData, setFormData] = useState({
    name: '',
    aboutText: '',
    coursesOffered: '',
    contactEmail: ''
  });

  // Fetching data from mongodb to show this visible on fronend
  const fetchData = () => {
    fetch('http://localhost:5000/api/college-info')
      .then((res) => res.json())
      .then((data) => {
        let collegeInfo = null;
        if (Array.isArray(data) && data.length > 0) collegeInfo = data[0];
        else if (data && data.name) collegeInfo = data;

        if (collegeInfo) {
```

```
        setCollegeData(collegeInfo);
        setFormData({
          name: collegeInfo.name || '',
          aboutText: collegeInfo.aboutText || '',
          coursesOffered: Array.isArray(collegeInfo.coursesOffered)
            ? collegeInfo.coursesOffered.join(', ')
            : collegeInfo.coursesOffered || '',
          contactEmail: collegeInfo.contactEmail || ''
        });
      }
      setLoading(false);
    })
    .catch((err) => {
      console.error("Error fetching data:", err);
      setLoading(false);
    });
  };

  useEffect(() => {
    fetchData();
  }, []);

  const handleInputChange = (e) => setFormData({ ...formData, [e.target.name]:
e.target.value });

  const handleSubmit = (e) => {
    e.preventDefault();
    fetch('http://localhost:5000/api/college-info', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
```

```
    body: JSON.stringify(formData),
  })
    .then((res) => res.json())
    .then((data) => {
      alert(data.message);
      fetchData();
      setIsEditing(false);
    })
    .catch((err) => console.error("Error saving data:", err));
};

if (loading) return <div style={styles.loader}>Loading Campus Data...</div>;

return (
  <div style={styles.container}>

    <nav style={styles.navbar}>
      <div style={styles.logo}>{collegeData ? collegeData.name : 'University Portal'}</div>
      <button
        onClick={() => setIsEditing(!isEditing)}
        style={isEditing ? styles.btnCancel : styles.btnEdit}
      >
        {isEditing ? 'Cancel Editing' : 'Admin Dashboard'}
      </button>
    </nav>

    {isEditing ? (
      // --- ADMIN FORM UI ---
      <div style={styles.adminWrapper}>
```

```
      <div style={styles.adminCard}>
        <h2 style={{ color: '#1e3a8a', marginBottom: '20px', borderBottom: '2px solid #f59e0b', paddingBottom: '10px' }}>Update College Database</h2>
        <form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>

          <div>
            <label style={styles.label}>College Name</label>
            <input type="text" name="name" value={formData.name} onChange={handleInputChange} required style={styles.input} />
          </div>

          <div>
            <label style={styles.label}>About the College</label>
            <textarea name="aboutText" value={formData.aboutText} onChange={handleInputChange} required rows="5" style={styles.input} />
          </div>

          <div>
            <label style={styles.label}>Courses Offered (Comma separated)</label>
            <input type="text" name="coursesOffered" value={formData.coursesOffered} onChange={handleInputChange} required placeholder="e.g. B.Tech, MBA, B.Sc" style={styles.input} />
          </div>

          <div>
            <label style={styles.label}>Contact Email</label>
            <input type="email" name="contactEmail" value={formData.contactEmail} onChange={handleInputChange} required style={styles.input} />
          </div>

          <button type="submit" style={styles.btnSubmit}>Save to MongoDB</button>
```

```
          </form>
        </div>
      </div>
    ) : (
      // UI which student can view
      <div>
        {collegeData ? (
          <>
            <header style={styles.hero}>
              <div style={styles.heroOverlay}>
                <h1 style={styles.heroTitle}>Welcome to {collegeData.name}</h1>
                <p style={styles.heroSubtitle}>Empowering the leaders of tomorrow through innovation and excellence.</p>
                <button style={styles.btnPrimary}>Apply Now</button>
              </div>
            </header>

            //About Section
            <section style={styles.section}>
              <div style={styles.aboutCard}>
                <h2 style={styles.sectionTitle}>Our Legacy</h2>
                <p style={styles.aboutText}>{collegeData.aboutText}</p>
              </div>
            </section>

            //Static section
            <section style={styles.statsSection}>
              <div style={styles.statBox}><h3>10,000+</h3><p>Students</p></div>
              <div style={styles.statBox}><h3>50+</h3><p>Programs</p></div>
              <div style={styles.statBox}><h3>95%</h3><p>Placement Rate</p></div>
```

```
          <div style={styles.statBox}><h3>200+</h3><p>Faculty Members</p></div>
        </section>

        //Courses Section
        <section style={{...styles.section, backgroundColor: '#f8fafc'}}>
          <h2 style={styles.sectionTitle}>Academic Programs</h2>
          <div style={styles.grid}>
            {collegeData.coursesOffered.map((course, index) => (
              <div key={index} style={styles.courseCard}>
                <span style={{ fontSize: '30px' }}></span>
                <h3 style={{ marginTop: '10px', color: '#1e3a8a' }}>{course}</h3>
                <p style={{ color: '#64748b', fontSize: '14px', marginTop: '10px' }}>Join our world-class faculty in mastering {course}.</p>
              </div>
            ))}
          </div>
        </section>

          // Static Faculties Section
         <section style={styles.section}>
          <h2 style={styles.sectionTitle}>Campus Life & Facilities</h2>
          <div style={styles.grid}>
            <div style={styles.facilityCard}> <strong>Advanced Labs</strong> - State-of-the-art equipment.</div>
            <div style={styles.facilityCard}><strong>Sports Complex</strong> - Indoor and outdoor arenas.</div>
            <div style={styles.facilityCard}><strong>Central Library</strong> - 24/7 access to millions of books.</div>
            <div style={styles.facilityCard}> <strong>Student Hostels</strong> - Comfortable campus living.</div>
          </div>
```

```
            </section>


            <footer style={styles.footer}>
              <div style={{ flex: 1 }}>
                <h3 style={{ color: '#f59e0b' }}>{collegeData.name}</h3>
                <p style={{ color: '#cbd5e1', marginTop: '10px' }}>Building the future, one student at a time.</p>
              </div>
              <div>
                <h4 style={{ color: 'white', marginBottom: '10px' }}>Contact Us</h4>
                <p><a href={`mailto:${collegeData.contactEmail}`} style={{ color: '#f59e0b', textDecoration: 'none' }}>{collegeData.contactEmail}</a></p>
                <p>123 University Avenue, Tech City</p>
              </div>
            </footer>
          </>
        ) : (
          <div style={styles.emptyState}>
            <h2>No campus data found in the database.</h2>
            <p>Click "Admin Dashboard" in the top right to set up the website!</p>
          </div>
        )}
      </div>
    )}
  </div>
 );
}
//CSS styles
const styles = {
```

```
  container: { fontFamily: "'Segoe UI', Roboto, Helvetica, Arial, sans-serif", margin: 0, padding: 0, backgroundColor: '#ffffff', color: '#333' },

  loader: { height: '100vh', display: 'flex', justifyContent: 'center', alignItems: 'center', fontSize: '24px', fontWeight: 'bold', color: '#1e3a8a' },


//Navigation bar

  navbar: { display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '15px 50px', backgroundColor: '#1e3a8a', color: 'white', position: 'sticky', top: 0, zIndex: 100, boxShadow: '0 4px 6px rgba(0,0,0,0.1)' },

  logo: { fontSize: '24px', fontWeight: 'bold', letterSpacing: '1px' },

  btnEdit: { padding: '10px 20px', backgroundColor: '#f59e0b', color: '#fff', border: 'none', borderRadius: '5px', cursor: 'pointer', fontWeight: 'bold', transition: '0.3s' },

  btnCancel: { padding: '10px 20px', backgroundColor: '#ef4444', color: '#fff', border: 'none', borderRadius: '5px', cursor: 'pointer', fontWeight: 'bold' },


  //Hero Section

  hero: {

    backgroundImage: 'url("https://images.unsplash.com/photo-1541339907198-e08756dedf3f?ixlib=rb-1.2.1&auto=format&fit=crop&w=1920&q=80")',

    backgroundSize: 'cover', backgroundPosition: 'center', height: '80vh', position: 'relative'

  },

  heroOverlay: { backgroundColor: 'rgba(15, 23, 42, 0.7)', height: '100%', width: '100%', display: 'flex', flexDirection: 'column', justifyContent: 'center', alignItems: 'center', textAlign: 'center', padding: '0 20px' },

  heroTitle: { color: 'white', fontSize: '56px', fontWeight: '800', marginBottom: '20px', textShadow: '2px 2px 4px rgba(0,0,0,0.5)' },

  heroSubtitle: { color: '#e2e8f0', fontSize: '24px', marginBottom: '30px', maxWidth: '800px' },

  btnPrimary: { padding: '15px 40px', backgroundColor: '#f59e0b', color: '#fff', border: 'none', borderRadius: '30px', fontSize: '18px', fontWeight: 'bold', cursor: 'pointer', boxShadow: '0 4px 15px rgba(245, 158, 11, 0.4)' },


  //Sections

  section: { padding: '80px 10%', textAlign: 'center' },
```

```
  sectionTitle: { fontSize: '36px', color: '#1e3a8a', marginBottom: '40px', fontWeight: '700' },

  aboutCard: { backgroundColor: 'white', padding: '40px', borderRadius: '15px', boxShadow: '0 10px 25px rgba(0,0,0,0.05)', maxWidth: '900px', margin: '0 auto' },

  aboutText: { fontSize: '18px', lineHeight: '1.8', color: '#475569' },


  statsSection: { display: 'flex', justifyContent: 'space-around', flexWrap: 'wrap', backgroundColor: '#1e3a8a', color: 'white', padding: '50px 10%' },

  statBox: { textAlign: 'center', minWidth: '150px', margin: '10px' },


//Grids for Courses and Faculties

  grid: { display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(250px, 1fr))', gap: '30px', maxWidth: '1200px', margin: '0 auto' },

  courseCard: { backgroundColor: 'white', padding: '30px', borderRadius: '10px', boxShadow: '0 4px 15px rgba(0,0,0,0.05)', textAlign: 'center', borderTop: '5px solid #f59e0b', transition: 'transform 0.3s' },

  facilityCard: { backgroundColor: '#f1f5f9', padding: '20px', borderRadius: '8px', fontSize: '16px', color: '#334155', textAlign: 'left', borderLeft: '4px solid #1e3a8a' },


  //Admin Form

  adminWrapper: { padding: '60px 20px', backgroundColor: '#f8fafc', minHeight: '80vh', display: 'flex', justifyContent: 'center' },

  adminCard: { backgroundColor: 'white', padding: '40px', borderRadius: '15px', boxShadow: '0 10px 30px rgba(0,0,0,0.1)', width: '100%', maxWidth: '600px' },

  label: { fontWeight: '600', color: '#334155', marginBottom: '5px', display: 'block' },

  input: { width: '100%', padding: '15px', borderRadius: '8px', border: '1px solid #cbd5e1', fontSize: '16px', boxSizing: 'border-box', backgroundColor: '#f8fafc' },

  btnSubmit: { padding: '15px', backgroundColor: '#10b981', color: 'white', border: 'none', borderRadius: '8px', fontSize: '18px', fontWeight: 'bold', cursor: 'pointer', marginTop: '10px' },


  // Footer

  footer: { display: 'flex', justifyContent: 'space-between', flexWrap: 'wrap', padding: '50px 10%', backgroundColor: '#0f172a', color: '#94a3b8' },
```

```
  emptyState: { textAlign: 'center', padding: '100px 20px', color: '#64748b' }
};
```

//Backend (Node.js/ Express.js)

Config > db.js

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
   try {
      await mongoose.connect(process.env.MONGO_URL);
      console.log('MongoDB connected');
   } catch (error) {
      console.error('Error connecting to MongoDB:', error);
      process.exit(1);
   }
}

module.exports = connectDB;
```

controllers > studentsubmit.js

```
async (req, res) => {
 try {
   const { name, aboutText, coursesOffered, contactEmail } = req.body;

   const coursesArray = typeof coursesOffered === 'string'
     ? coursesOffered.split(',').map(course => course.trim())
     : coursesOffered;
```

```
    const updatedInfo = await CollegeInfo.findOneAndUpdate(
      {},
      { name, aboutText, coursesOffered: coursesArray, contactEmail },
      { new: true, upsert: true }
    );

    res.json({ message: "College data saved to MongoDB successfully!", data: updatedInfo
});
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Server Error saving data" });
  }
}
```

models > collegeSchema.js

```
const mongoose = require('mongoose');

const collegeSchema = new mongoose.Schema({
  name: String,
  aboutText: String,
  coursesOffered: [String],
  contactEmail: String
});

module.exports = mongoose.model('College', collegeSchema);
```

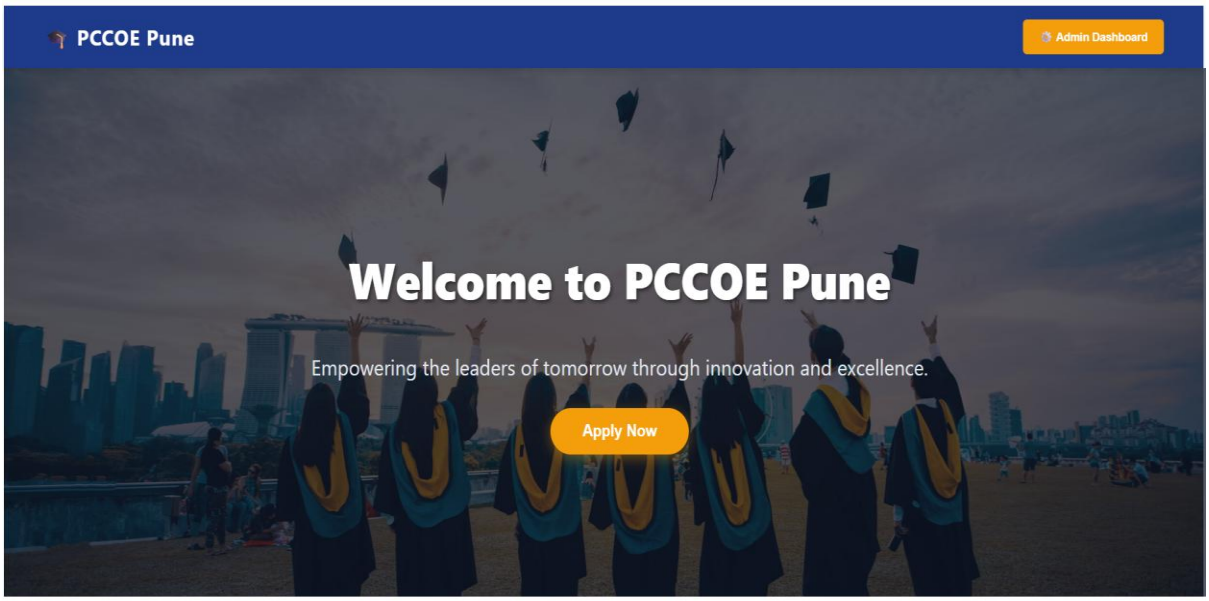
server.js

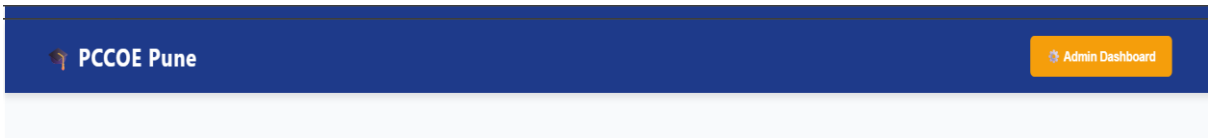
```
const express = require('express');
const connectDB = require('./config/db')
const studentRouter = require('./router/studentRouter');
const CollegeInfo = require('./models/CollegeSchema');
const cors  = require('cors');
```

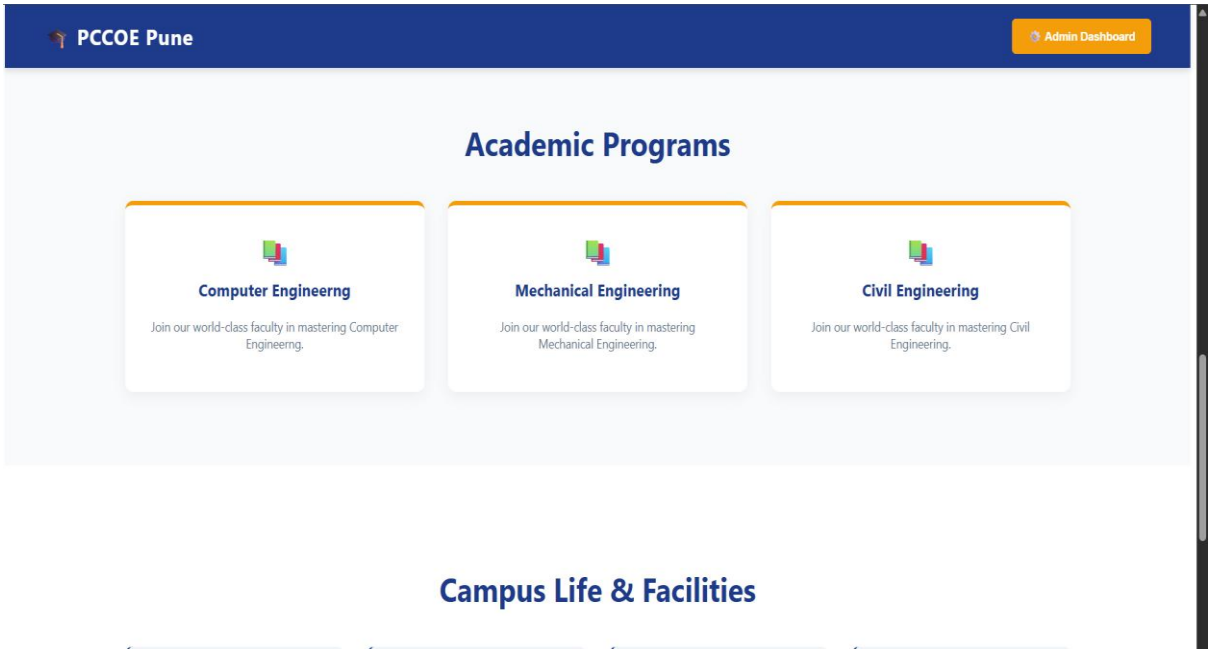
```
const dotenv = require('dotenv');
dotenv.config();

const app = express();
app.use(cors());
app.use(express.json());

const PORT = process.env.PORT || 5000;

connectDB();

// Register the router for GET endpoint
app.use('/api', studentRouter);

app.post('/api/college-info', async (req, res) => {
  try {
    const { name, aboutText, coursesOffered, contactEmail } = req.body;

    const coursesArray = typeof coursesOffered === 'string'
      ? coursesOffered.split(',').map(course => course.trim())
      : coursesOffered;

    const updatedInfo = await CollegeInfo.findOneAndUpdate(
      {},
      { name, aboutText, coursesOffered: coursesArray, contactEmail },
      { new: true, upsert: true }
    );

    res.json({ message: "College data saved to MongoDB successfully!", data: updatedInfo
});
```

```
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Server Error saving data" });
  }
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

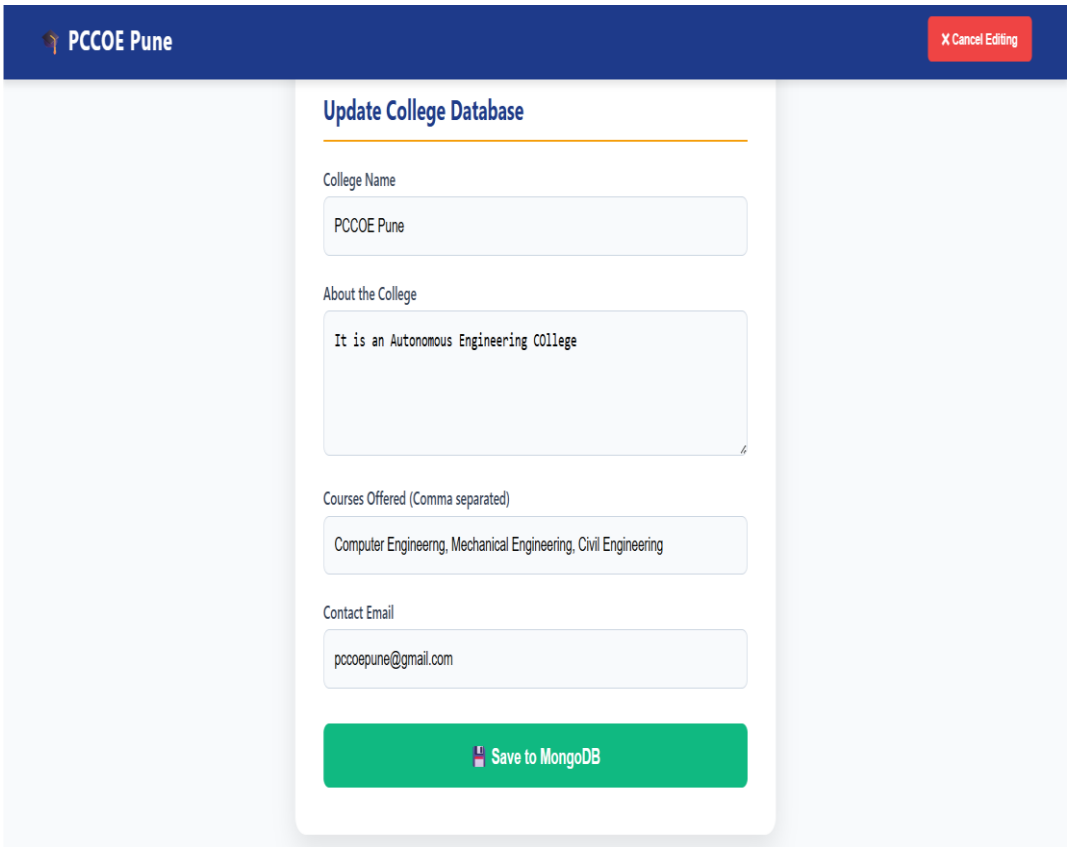
Screenshots of Working Website







Campus Life & Facilities



Conclusion

The project successfully demonstrates a working, full-stack web application. By utilizing the MERN stack (specifically focusing on Node, Express, and Mongo for the backend), we created a highly performant and scalable platform.

The implementation of a unified frontend that serves both as the public-facing college portal and the administrative data-entry dashboard showcases an efficient use of React state management. Future scalability could involve adding secure JWT authentication for the admin panel, expanding the database to include individual student portals, and integrating cloud storage for campus images.